Тестирање на софтвер

Извештај за лабораториска вежба 2

Интеграциски Тестови

1. **Поставување на околина** – во двата проекти во pom.xml фајлот требаше да додадам dependencies и plugins:

Во **pom.xml** на **booking-service** – додадов **2 dependencies** и **2 plugins**:

Dependencies:

- **spring-boot-starter-webmvc-test** – овој dependency ги вклучува Junit и Mockito тестовите и овозможува тестирање на HTTP API-то на booking-service.
- **spring-boot-starter-data-jpa-test** – овој dependency овозможува тестирање на repository со вистинска база.

Plugins:

- **maven-dependency-plugin** – овој plugin претставува патека до Mockito.
- **maven-failsafe-plugin** – овој plugin ги извршува интеграциските тестови преку командата sh **mvnw verify**.

Во **pom.xml** на **membership-registry** – додадов **1 dependency** и **1 plugin**:

Dependency:

- **spring-boot-starter-webmvc-test** – овој dependency ги вклучува Junit и Mockito тестовите и овозможува тестирање на HTTP API-то на membership-registry.

Plugin:

- **maven-failsafe-plugin** – овој plugin ги извршува интеграциските тестови преку командата sh **mvnw verify**.

По извршување на командата **sh mvnw verify**, добиваме successful build што значи дека околината е точно направена:

```
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for Workshop Booking - Integration Testing Homework 1.0.0:
[INFO]
[INFO] Workshop Booking - Integration Testing Homework .... SUCCESS [  0.000 s]
[INFO] membership-registry ................................ SUCCESS [  0.593 s]
[INFO] booking-service .................................... SUCCESS [  0.322 s]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.015 s
[INFO] Finished at: 2026-11-09T15:59:49+01:00
[INFO] ------------------------------------------------------------------------
```

– sh mvnw verify

Исто така креирав и **application-test.properties** фајл во **booking-service/src/test/resources** папката, кој што се активира во тестовите со анотацијата **@ActiveProfiles("test")** и **ги заменува вредностите од application.properties фајлот** во текот на тестирањето:

spring.datasource.url=jdbc:h2:mem:booking-${random.uuid};DB_CLOSE_DELAY=-1
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=create-drop
registry.base-url=http://localhost:1
registry.connect-timeout-ms=500
registry.read-timeout-ms=500

2. **Идентификување компоненти и меѓусебни зависности**

Идентификувајте ги компонентите и нивните меѓусебни зависности	
Компонента	**Зависност**
BookingController	BookingService
BookingService	WorkshopRepository, RegistrationRepository & MembershipGateway
WorkshopRepository & RegistrationRepository	Workshop & Registration
Workshop & Registration	H2 / PostgreSQL Database
H2 / PostgreSQL Database	**НЕМА ЗАВИСНОСТ**
ApiErrors	BookingFailure & DataIntegrityViolationException
HttpMembershipGateway	membership-registry апликацијата
MemberController	**НЕМА ЗАВИСНОСТ**

Компонентите ги имаат овие зависности поради тоа што ова е SpringBoot проект и хиерархијата на фајловите во SpringBoot проектите изгледа вака: **Controller → Service → Repository → Entity → DB** (Исто како изучениот материјал на предметот Веб Програмирање)

– Испланирајте кои интеграции ќе ги тестирате – тука би користел **bottom up стратегија** за интеграциски тестови, односно **прво би почнал од најдолните компоненти** и би се движел нагоре. Планот ми е следниот:

1. **Тестирање на DB и Repository** – дали податоците во нив се зачувани и дали работат ограничувањата во нив.
2. **Тестирање на Service + Repository/DB** – дали регистрацијата на податоците правилно се извршува во DB.
3. **Тестирање на трансакции** – дали ако една пријава во групата е невалидна, не се зачувува ниедна пријава од таа група на пријави.

4. **Тестирање на Controller** – дали HTTP барањето враќа правилен статус и JSON одговор и дали состојбата која што се чува во базата е точна**.**
5. **Тестирање на HttpMembershipGateway** – дали клиентот правилно ги обработува одговорите, грешките и timeout-ите од membership-registry.
6. **Тестирање на membership-registry** – дали membership-registry враќа JSON во очекуваниот формат од страна на booking-service.

– Кои компоненти ќе бидат реални и каде е соодветно да користите замени – компонентите што се тестираат се **секогаш реални** (Controller, Service, Repository, Entity и DB). Repository-то **НЕ се заменува со mock**, бидејќи тогаш не би се проверило **дали податоците навистина се зачувуваат во базата** (разликата меѓу Mock тестови и Интеграциски тестови). Замени се користат само за **membership-registry** (надворешен сервис). Во тестовите за **Service и Controller**, membership-registry се заменува со **stub** (@MockitoBean). Во тестот за **HttpMembershipGateway** се заменува со FakeRegistry, што претставува локален HTTP сервер што може да симулира грешки и timeout.

3. **Интеграциски тестови што ја проверуваат соработката меѓу компонентите**

Тест 1 – RegistrationRepositoryIT (Тестирање на DB и Repository) – тука опфатив: **успешно сценарио**, **неуспешно сценарио** и **работа со базата на податоци**. Напишав **5 теста** и тие се следните:

- **saveAndLoadRegistration** – **успешно** зачувување во база и **читање на истите податоци**

```
@Test  no usages  new *
void saveAndLoadRegistration() {
    Registration saved = registrations.saveAndFlush(
            new Registration("REF-1", "Ana", "ana@example.com", "C-01", 2, java));
    em.clear();

    Registration loaded = registrations.findByReference("REF-1").orElseThrow();
    assertEquals(saved.getId(), loaded.getId());
    assertEquals("Ana", loaded.getAttendee());
    assertEquals("ana@example.com", loaded.getEmail());
    assertEquals(2, loaded.getTickets());
    assertEquals("JAVA-01", loaded.getWorkshop().getCode());
}
```

- **rejectDuplicateReference** – **одбиено** барање до базата за да се зачува пријава со **ист reference**

```
@Test  no usages  new *
void rejectDuplicateReference() {
    registrations.saveAndFlush(new Registration("REF-1", "Ana", "ana@example.com", "C-01", 1, java));

    assertThrows(DataIntegrityViolationException.class, () -> registrations.saveAndFlush(new Registration("REF-1", "Boris", "boris@example.com", "C-01", 1, java)));
}
```

- **rejectSameEmailInSameWorkshop** – **одбиено** барање до базата за да се зачуваат **2 пријави со ист email во иста работилница**

```
@Test  no usages  new *
void rejectSameEmailInSameWorkshop() {
    registrations.saveAndFlush(new Registration("REF-1", "Ana", "ana@example.com", "C-01", 1, java));

    assertThrows(DataIntegrityViolationException.class, () -> registrations.saveAndFlush(new Registration("REF-2", "Ana", "ana@example.com", "C-01", 1, java)));
}
```

- **rejectDuplicateWorkshopCode** – **одбиено** барање до базата за да се зачува **работилница со код кој што е ист како кодот во setUp()**

```
@Test  no usages  new *
void rejectDuplicateWorkshopCode() {
    assertThrows(DataIntegrityViolationException.class, () -> workshops.saveAndFlush(new Workshop("JAVA-01", "Copy", true)));
}
```

- **findByWorkshopCodeAndAttendeePrefix** – **успешно** пребарување по **код на работилница** и **почеток на име**, кое што враќа **само точни пријави**.

```
@Test  no usages  new *
void findByWorkshopCodeAndAttendeePrefix() {
    Workshop python = workshops.saveAndFlush(new Workshop("PY-01", "Python Workshop", true));
    registrations.saveAndFlush(new Registration("REF-1", "Ana", "ana@example.com", "C-01", 1, java));
    registrations.saveAndFlush(new Registration("REF-2", "Andrej", "andrej@example.com", "C-01", 1, java));
    registrations.saveAndFlush(new Registration("REF-3", "Boris", "boris@example.com", "C-01", 1, java));
    registrations.saveAndFlush(new Registration("REF-4", "Anita", "anita@example.com", "C-01", 1, python));
    em.clear();

    Page<Registration> page = registrations.findByWorkshopCodeAndAttendeeStartingWith("JAVA-01", "An", PageRequest.of(0, 10));

    assertEquals(2, page.getTotalElements());
}
```

По извршување на командата **sh mvnw -pl booking-service verify**, **5те тестови** успешно поминаа:

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.426 s -- in mk.ukim.finki.testing.booking.RegistrationRepositoryIT
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- failsafe:3.5.6:verify (default) @ booking-service ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.067 s
[INFO] Finished at: 2026-11-10T00:48:21+01:00
[INFO] ------------------------------------------------------------------------
```

Тест 2 – BookingServiceIT (Тестирање на Service + Repository/DB) – тука опфатив: успешно сценарио, неуспешно сценарио и работа со базата на податоци. Напишав **6 теста** и тие се следните:

- **registerAndSaveNormalizedData** – **успешна** регистрација преку Service и **зачувување во база**, при што **името се зачувува без празни места**, а **email-от со мали букви**

```
@Test  no usages  new *
void registerAndSaveNormalizedData() {
    service.register(new BookingRequest("REF-1", "JAVA-01", "  Ana  ", "ANA@Example.com", "C-01", 2));
    em.flush();
    em.clear();

    Registration loaded = registrations.findByReference("REF-1").orElseThrow();
    assertEquals("Ana", loaded.getAttendee());
    assertEquals("ana@example.com", loaded.getEmail());
    assertEquals(2, loaded.getTickets());
    assertEquals("JAVA-01", loaded.getWorkshop().getCode());
}
```

- **findSavedRegistration** – **успешно** пребарување на зачувана пријава **преку Service**

```
@Test  no usages  new *
void findSavedRegistration() {
    service.register(new BookingRequest("REF-1", "JAVA-01", "Ana", "ana@example.com", "C-01", 1));
    em.flush();
    em.clear();

    RegistrationView view = service.find("REF-1");
    assertEquals("Ana", view.attendee());
    assertEquals("JAVA-01", view.workshopCode());
}
```

- **rejectInvalidInput** – **одбиено** барање за пријава со **невалиден влез** (0 билети), пред да се повика registry и без да се зачува ништо во базата

```
@Test  no usages  new *
void rejectInvalidInput() {
    BookingFailure failure = assertThrows(BookingFailure.class, () -> service.register(
            new BookingRequest("REF-1", "JAVA-01", "Ana", "ana@example.com", "C-01", 0)));

    assertEquals("INVALID_INPUT", failure.getCode());
    assertEquals(0, registrations.count());
    verify(membership, never()).isEligible(anyString());
}
```

- **rejectUnknownWorkshop** – **одбиено** барање за пријава во **работилница која што не постои**

```
@Test  no usages  new *
void rejectUnknownWorkshop() {
    BookingFailure failure = assertThrows(BookingFailure.class, () -> service.register(
            new BookingRequest("REF-1", "NONE-01", "Ana", "ana@example.com", "C-01", 1)));

    assertEquals("WORKSHOP_NOT_FOUND", failure.getCode());
    assertEquals(0, registrations.count());
}
```

- **rejectClosedWorkshop** – **одбиено** барање за пријава во **работилница која што е затворена**

```
@Test  no usages  new *
void rejectClosedWorkshop() {
    BookingFailure failure = assertThrows(BookingFailure.class, () -> service.register(
            new BookingRequest("REF-1", "CLOSED-01", "Ana", "ana@example.com", "C-01", 1)));

    assertEquals("WORKSHOP_CLOSED", failure.getCode());
    assertEquals(0, registrations.count());
}
```

- **rejectIneligibleMember** – **одбиено** барање за пријава од **член кој што не е валиден** во registry (без да се зачува ништо во базата)

```
@Test  no usages  new *
void rejectIneligibleMember() {
    when(membership.isEligible("C-02")).thenReturn(false);

    BookingFailure failure = assertThrows(BookingFailure.class, () -> service.register(
            new BookingRequest("REF-1", "JAVA-01", "Ana", "ana@example.com", "C-02", 1)));

    assertEquals("MEMBER_INELIGIBLE", failure.getCode());
    assertEquals(0, registrations.count());
}
```

По извршување на командата **sh mvnw -pl booking-service verify**, **11те** (5те од претходно и 6те нови) **тестови** успешно поминаа:

```
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- failsafe:3.5.6:verify (default) @ booking-service ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.824 s
[INFO] Finished at: 2026-11-10T01:14:52+01:00
[INFO] ------------------------------------------------------------------------
```

Тест 3 – BookingTransactionIT (Тестирање на трансакции) – тука опфатив: **успешно сценарио**, **неуспешно сценарио**, **работа со базата на податоци** и **трансакции**. Напишав **5 теста** и тие се следните:

- **saveWholeBatch** – **успешно** зачувување на **групна пријава од 3 пријави** во база

```
@Test
void saveWholeBatch() {
    service.registerBatch(List.of(
            request("REF-1", "JAVA-01", "ana@example.com", "C-01"),
            request("REF-2", "JAVA-01", "boris@example.com", "C-01"),
            request("REF-3", "JAVA-01", "vesna@example.com", "C-01")));

    assertEquals(3, registrations.count());
}
```

- **rollbackBatchWhenWorkshopIsClosed** – **одбиено** барање за групна пријава каде **една пријава е за затворена работилница** (ниедна пријава од групата не се зачувува во базата, **трансакции**)

```
@Test  no usages  new *
void rollbackBatchWhenWorkshopIsClosed() {
    BookingFailure failure = assertThrows(BookingFailure.class, () -> service.registerBatch(List.of(
            request("REF-1", "JAVA-01", "ana@example.com", "C-01"),
            request("REF-2", "JAVA-01", "boris@example.com", "C-01"),
            request("REF-3", "CLOSED-01", "vesna@example.com", "C-01"))));

    assertEquals("WORKSHOP_CLOSED", failure.getCode());
    assertEquals(0, registrations.count());
}
```

- **rollbackBatchWhenMemberIsIneligible** – **одбиено** барање за групна пријава каде **еден член не е валиден во registry** (ниедна пријава од групата не се зачувува во базата, **трансакции**)

```
@Test  no usages  new *
void rollbackBatchWhenMemberIsIneligible() {
    when(membership.isEligible("C-02")).thenReturn(false);

    BookingFailure failure = assertThrows(BookingFailure.class, () -> service.registerBatch(List.of(
            request("REF-1", "JAVA-01", "ana@example.com", "C-01"),
            request("REF-2", "JAVA-01", "boris@example.com", "C-02"))));

    assertEquals("MEMBER_INELIGIBLE", failure.getCode());
    assertEquals(0, registrations.count());
}
```

- **rollbackBatchWhenReferenceIsDuplicate** – **одбиено** барање до базата за групна пријава со **2 пријави со ист reference** (ниедна пријава од групата не се зачувува во базата, **трансакции**)

```
@Test  no usages  new *
void rollbackBatchWhenReferenceIsDuplicate() {
    assertThrows(DataIntegrityViolationException.class, () -> service.registerBatch(List.of(
            request("REF-1", "JAVA-01", "ana@example.com", "C-01"),
            request("REF-1", "JAVA-01", "boris@example.com", "C-01"))));

    assertEquals(0, registrations.count());
}
```

- **rejectTooLargeBatch** – **одбиено** барање за групна **пријава со повеќе од 10 пријави** (без да се зачува ништо во базата)

```
@Test  no usages  new *
void rejectTooLargeBatch() {
    List<BookingRequest> requests = new ArrayList<>();
    for (int i = 1; i <= 11; i++) {
        requests.add(request("REF-" + i, "JAVA-01", "user" + i + "@example.com", "C-01"));
    }

    BookingFailure failure = assertThrows(BookingFailure.class, () -> service.registerBatch(requests));

    assertEquals("INVALID_INPUT", failure.getCode());
    assertEquals(0, registrations.count());
}
```

По извршување на командата **sh mvnw -pl booking-service verify**, **16те** (11те од претходно и 5те нови) **тестови** успешно поминаа:

```
[INFO] Results:
[INFO]
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- failsafe:3.5.6:verify (default) @ booking-service ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.873 s
[INFO] Finished at: 2026-11-10T01:36:27+01:00
[INFO] ------------------------------------------------------------------------
```

Тест 4 – BookingApiIT (Тестирање на Controller) – тука опфатив: **успешно сценарио**, **неуспешно сценарио**, **работа со базата на податоци**, **HTTP комуникација** и **договори за размена на податоци**. Напишав **9 теста** и тие се следните:

- **createRegistration** – **успешна** регистрација **преку HTTP барање** (201 и JSON одговор), при што **пријавата е зачувана во базата**

```
@Test
void createRegistration() throws Exception {
    mvc.perform(post("/registrations").contentType("application/json")
                    .content(json("REF-1", "JAVA-01", "C-01", 2)))
            .andExpect(status().isCreated())
            .andExpect(header().string("Location", "/registrations/REF-1"))
            .andExpect(jsonPath("$.reference").value("REF-1"))
            .andExpect(jsonPath("$.attendee").value("Ana"));

    em.flush();
    em.clear();
    Registration loaded = registrations.findByReference("REF-1").orElseThrow();
    assertEquals("Ana", loaded.getAttendee());
    assertEquals("ana@example.com", loaded.getEmail());
}
```

- **getRegistration** – **успешно** пребарување на **пријава преку HTTP барање**, при што **JSON одговорот ги содржи сите полиња со точни типови** (договори за размена на податоци)

```
@Test
void getRegistration() throws Exception {
    mvc.perform(post("/registrations").contentType("application/json")
            .content(json("REF-1", "JAVA-01", "C-01", 2)));
    em.flush();
    em.clear();

    mvc.perform(get("/registrations/REF-1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").isNumber())
            .andExpect(jsonPath("$.reference").isString())
            .andExpect(jsonPath("$.workshopCode").value("JAVA-01"))
            .andExpect(jsonPath("$.attendee").isString())
            .andExpect(jsonPath("$.email").isString())
            .andExpect(jsonPath("$.tickets").isNumber());
}
```

- **rejectInvalidInput** – **одбиено** барање за пријава со **невалиден влез** (0 билети) со **статус 400** (без да се зачува ништо во базата)

```
@Test
void rejectInvalidInput() throws Exception {
    mvc.perform(post("/registrations").contentType("application/json")
                    .content(json("REF-1", "JAVA-01", "C-01", 0)))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.code").value("INVALID_INPUT"));

    assertEquals(0, registrations.count());
}
```

- **rejectInvalidJson** – **одбиено** барање со **неисправен JSON** со **статус 400**

```
@Test  no usages  new *
void rejectInvalidJson() throws Exception {
    mvc.perform(post("/registrations").contentType("application/json")
                    .content("{\"reference\":"))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.code").value("INVALID_INPUT"));
}
```

- **rejectClosedWorkshop** – **одбиено** барање за пријава во **работилница која што е затворена** со **статус 409** (без да се зачува ништо во базата)

```
@Test
void rejectClosedWorkshop() throws Exception {
    mvc.perform(post("/registrations").contentType("application/json")
                    .content(json("REF-1", "CLOSED-01", "C-01", 1)))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("WORKSHOP_CLOSED"));

    assertEquals(0, registrations.count());
}
```

- **rejectIneligibleMember** – **одбиено** барање за пријава од **член кој што не е валиден во registry** со **статус 422** (без да се зачува ништо во базата)

```
@Test
void rejectIneligibleMember() throws Exception {
    when(membership.isEligible("C-02")).thenReturn(false);

    mvc.perform(post("/registrations").contentType("application/json")
                    .content(json("REF-1", "JAVA-01", "C-02", 1)))
            .andExpect(status().isUnprocessableContent())
            .andExpect(jsonPath("$.code").value("MEMBER_INELIGIBLE"));

    assertEquals(0, registrations.count());
}
```

- **rejectWhenRegistryIsUnavailable** – **одбиено** барање за **пријава кога registry не е достапен** со **статус 503** (без да се зачува ништо во базата)

```
@Test
void rejectWhenRegistryIsUnavailable() throws Exception {
    when(membership.isEligible("C-01")).thenThrow(new BookingFailure(503, "REGISTRY_UNAVAILABLE"));

    mvc.perform(post("/registrations").contentType("application/json")
                    .content(json("REF-1", "JAVA-01", "C-01", 1)))
            .andExpect(status().isServiceUnavailable())
            .andExpect(jsonPath("$.code").value("REGISTRY_UNAVAILABLE"));

    assertEquals(0, registrations.count());
}
```

- **rejectDuplicateReference** – **одбиено** барање до базата за да се зачува **пријава со ист reference** со **статус 409**

```
@Test
void rejectDuplicateReference() throws Exception {
    mvc.perform(post("/registrations").contentType("application/json")
                    .content(json("REF-1", "JAVA-01", "C-01", 1)))
            .andExpect(status().isCreated());

    mvc.perform(post("/registrations").contentType("application/json")
                    .content(json("REF-1", "JAVA-01", "C-01", 1).replace("ANA@", "BORIS@")))
            .andExpect(status().isConflict())
            .andExpect(jsonPath("$.code").value("DATA_CONFLICT"));
}
```

- **returnNotFoundForUnknownReference** – **одбиено** барање за пребарување на **пријава која што не постои** со **статус 404**

```
@Test
void returnNotFoundForUnknownReference() throws Exception {
    mvc.perform(get("/registrations/NONE-1"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.code").value("REGISTRATION_NOT_FOUND"));
}
```

По извршување на командата **sh mvnw -pl booking-service verify**, **25те** (16те од претходно и 9те нови) **тестови** успешно поминаа:

```
[INFO] Results:
[INFO]
[INFO] Tests run: 25, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- failsafe:3.5.6:verify (default) @ booking-service ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  4.174 s
[INFO] Finished at: 2026-11-10T01:59:55+01:00
[INFO] ------------------------------------------------------------------------
```

Тест 5 – HttpMembershipGatewayIT (Тестирање на HttpMembershipGateway) – тука опфатив: **успешно сценарио**, **неуспешно сценарио**, **HTTP комуникација** и **договори за размена на податоци**. Прво направив нов фајл **FakeRegistry**, за да можам **да ги тестирам статусите и одговорите од membership-registry** (надворешниот сервис). FakeRegistry служи како **локален HTTP сервер** што **го заменува вистинскиот membership-registry**. Во секој тест одбирам **очекуван статус**, **кој JSON ќе го врати** и **колку ќе доцни одговорот**. Ова **не е можно со вистинскиот registry**, бидејќи така **секогаш ќе добиваме исправни одговори**. Пред секој тест **FakeRegistry се враќа на нормален одговор**, а **на крајот се гаси**. Напишав **9 теста** и тие се следните:

- **eligibleMember** – **успешно** HTTP барање до registry **за валиден член**, при што барањето оди на точната патека **/members/C-01**

```
@Test  no usages  new *
void eligibleMember() {
    assertTrue(gateway.isEligible("C-01"));
    assertEquals("/members/C-01", registry.lastPath());
    assertEquals(1, registry.calls());
}
```

- **ineligibleMember** – **успешно** HTTP барање до registry за **член кој што не е валиден**

```
@Test  no usages  new *
void ineligibleMember() {
    registry.respond(200, "{\"memberId\":\"{id}\",\"eligible\":false}");

    assertFalse(gateway.isEligible("C-02"));
}
```

- **memberNotFound** – **одбиено** барање за **член кој што не посто**и во registry со **статус 404** (со грешка **MEMBER_NOT_FOUND**)

```
@Test  no usages  new *
void memberNotFound() {
    registry.respond(404, "{\"code\":\"MEMBER_NOT_FOUND\"}");

    BookingFailure failure = assertThrows(BookingFailure.class, () -> gateway.isEligible("C-99"));
    assertEquals("MEMBER_NOT_FOUND", failure.getCode());
    assertEquals(422, failure.getStatus());
}
```

- **registryServerError** – **одбиено** барање кога registry враќа **серверска грешка** со **статус 500** (со грешка **REGISTRY_CONTRACT_ERROR**)

```
@Test  no usages  new *
void registryServerError() {
    registry.respond(500, "{}");

    BookingFailure failure = assertThrows(BookingFailure.class, () -> gateway.isEligible("C-01"));
    assertEquals("REGISTRY_CONTRACT_ERROR", failure.getCode());
    assertEquals(502, failure.getStatus());
}
```

- **wrongMemberIdInResponse** – **одбиен** одговор од registry кој што е за **друг член** (договори за размена на податоци)

```
@Test  no usages  new *
void wrongMemberIdInResponse() {
    registry.respond(200, "{\"memberId\":\"C-02\",\"eligible\":true}");

    BookingFailure failure = assertThrows(BookingFailure.class, () -> gateway.isEligible("C-01"));
    assertEquals("REGISTRY_CONTRACT_ERROR", failure.getCode());
}
```

- **wrongEligibleType** – **одбиен** одговор од registry каде **полето eligible е од погрешен тип** (yes наместо true – договори за размена на податоци)

```
@Test  no usages  new *
void wrongEligibleType() {
    registry.respond(200, "{\"memberId\":\"{id}\",\"eligible\":\"yes\"}");

    BookingFailure failure = assertThrows(BookingFailure.class, () -> gateway.isEligible("C-01"));
    assertEquals("REGISTRY_CONTRACT_ERROR", failure.getCode());
}
```

- **wrongJsonFormat** – **одбиен** одговор од registry со **неисправен JSON формат** (фалат податоци – договори за размена на податоци)

```
@Test  no usages  new *
void wrongJsonFormat() {
    registry.respond(200, "{\"memberId\":");

    BookingFailure failure = assertThrows(BookingFailure.class, () -> gateway.isEligible("C-01"));
    assertEquals("REGISTRY_CONTRACT_ERROR", failure.getCode());
}
```

- **registryTimeout** – **одбиено** барање кога **registry одговара преспоро** (timeout – се претвора во грешка **REGISTRY_UNAVAILABLE**)

```
@Test  no usages  new *
void registryTimeout() {
    registry.delay(1500);

    BookingFailure failure = assertThrows(BookingFailure.class, () -> gateway.isEligible("C-01"));
    assertEquals("REGISTRY_UNAVAILABLE", failure.getCode());
    assertEquals(503, failure.getStatus());
}
```

- **registryUnavailable** – **одбиено** барање кога **registry не е вклучен** (со грешка **REGISTRY_UNAVAILABLE**)

```
@Test  no usages  new *
void registryUnavailable() {
    HttpMembershipGateway offline = new HttpMembershipGateway("http://localhost:1", 500, 500);

    BookingFailure failure = assertThrows(BookingFailure.class, () -> offline.isEligible("C-01"));
    assertEquals("REGISTRY_UNAVAILABLE", failure.getCode());
}
```

По извршување на командата **sh mvnw -pl booking-service verify**, **34те** (25те од претходно и 9те нови) **тестови** успешно поминаа:

```
[INFO] Results:
[INFO]
[INFO] Tests run: 34, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- failsafe:3.5.6:verify (default) @ booking-service ---
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.369 s
[INFO] Finished at: 2026-11-10T17:20:26+01:00
[INFO] ------------------------------------------------------------------------
```

Тест 6 – MemberControllerIT (Тестирање на membership-registry) – тука опфатив: **успешно сценарио**, **неуспешно сценарио**, **HTTP комуникација** и **договори за размена на податоци**. Овој тест е **во membership-registry** проектот и се тестира на вистинскиот registry. Со него **проверувам дали вистинскиот registry го враќа истиот JSON формат што го имитира FakeRegistry** во **тест 5**, со што се потврдува дека замената е исправна. Напишав **3 теста** и тие се следните:

- **eligibleMemberContract** – **успешно** HTTP барање за **валиден член** (C-01), при што **JSON одговорот ги содржи полињата memberId** и **eligible** со **точни типови** (договори за размена на податоци)

```
@Test
void eligibleMemberContract() throws Exception {
    mvc.perform(get("/members/C-01"))
            .andExpect(status().isOk())
            .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
            .andExpect(jsonPath("$.memberId").isString())
            .andExpect(jsonPath("$.memberId").value("C-01"))
            .andExpect(jsonPath("$.eligible").isBoolean())
            .andExpect(jsonPath("$.eligible").value(true));
}
```

- **ineligibleMemberContract** – **успешно** HTTP барање за **член кој што не е валиден** (C-02), при што **eligible е false** (договори за размена на податоци)

```
@Test
void ineligibleMemberContract() throws Exception {
    mvc.perform(get("/members/C-02"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.memberId").value("C-02"))
            .andExpect(jsonPath("$.eligible").isBoolean())
            .andExpect(jsonPath("$.eligible").value(false));
}
```

- **unknownMemberContract** – **одбиено** барање за **член кој што не постои** со **статус 404** (со грешка **MEMBER_NOT_FOUND**)

```
@Test
void unknownMemberContract() throws Exception {
    mvc.perform(get("/members/C-99"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.code").value("MEMBER_NOT_FOUND"));
}
```

По извршување на командата **sh mvnw verify**, **двата проекти успешно поминуваат** (**3 теста** во **membership-registry** и **34 теста** во **booking-service**).

```
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for Workshop Booking - Integration Testing Homework 1.0.0:
[INFO]
[INFO] Workshop Booking - Integration Testing Homework .... SUCCESS [  0.001 s]
[INFO] membership-registry ................................ SUCCESS [  1.860 s]
[INFO] booking-service .................................... SUCCESS [  3.912 s]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.890 s
[INFO] Finished at: 2026-11-10T17:48:00+01:00
[INFO] ------------------------------------------------------------------------
```

4. **Потребно е тестовите да бидат независни и повторливи** – **секој тест сам ги креира податоците што му требаат** при извршување и **по неговото завршување состојбата се враќа на почетна**, за да не зависи од другите тестови ниту од редоследот на извршување. Исто така **тестовите користат H2 база во меморија**, која **се креира пазна при секое стартување на тестовите** и **базата се брише на крајот**. По **извршување на тестовите 2 пати** едно подруго **го добиваме истиот резултат**, што докажува дека тие се **повторливи**:

```
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for Workshop Booking - Integration Testing Homework 1.0.0:
[INFO]
[INFO] Workshop Booking - Integration Testing Homework .... SUCCESS [  0.001 s]
[INFO] membership-registry ................................ SUCCESS [  1.854 s]
[INFO] booking-service .................................... SUCCESS [  4.060 s]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  6.033 s
[INFO] Finished at: 2026-11-10T18:12:30+01:00
[INFO] ------------------------------------------------------------------------
```

```
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for Workshop Booking - Integration Testing Homework 1.0.0:
[INFO]
[INFO] Workshop Booking - Integration Testing Homework .... SUCCESS [  0.001 s]
[INFO] membership-registry ................................ SUCCESS [  1.548 s]
[INFO] booking-service .................................... SUCCESS [  3.647 s]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.293 s
[INFO] Finished at: 2026-11-10T18:13:18+01:00
[INFO] ------------------------------------------------------------------------
```

Студент: Огнен Младеновски 233108